CDA 4203/4203L Spring 2026
Computer System Design -- Final Project
Prototyping of an Audio Message Recorder

Demo: Put the link to your demonstration Video (on YouTube, etc.) at the beginning of your report. Make sure you show as much as you could achieve in the Video.
Posted: March 27th, 2026, Report Deadline: 11:59PM, May 4th, 2026

This project carries 25% of the lecture grade and 50% of the lab grade.
You have a bit more than one month time.

Overview

In this project, you will prototype an audio recorder/player using the FPGA Board. An audio recorder/player system can be used in different applications including public health & safety, cultural, social events, etc.

The player must be implemented with picoBlaze soft microcontroller as the main controller. The design should have the standard features of an audio recorder, namely, ability to record an audio message, play/pause/delete an audio message selected by the user from the audio library. The user interface is through the Serial Terminal, push buttons, LEDs, etc.

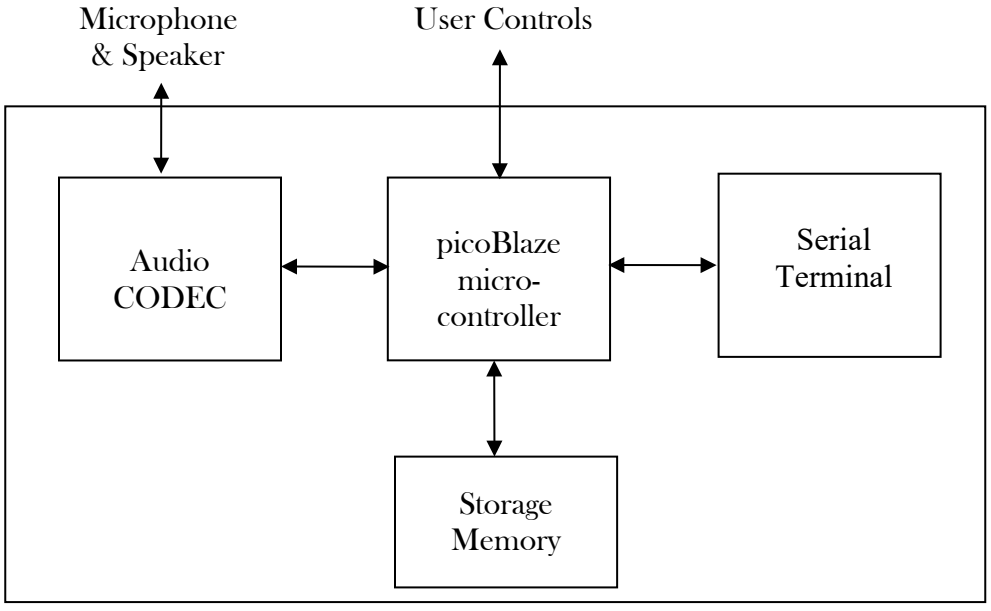

Figure 1: High-level block diagram of audio recorder/player

Contact
The lab TAs will be the main contacts.

Design Requirements

The following are the design requirements:

R1) On start up, the system must show a welcome message and then display a menu

1) Play a message
2) Record a message
3) Delete a message
4) Delete all messages
5) Volume control

The experience gained in Lab6, will help you here. Basically, like Lab6, you can use the ASCII characters to create the above menu.

R2) While playing an audio message, the user be able to pause/resume the message.

R3) The recorder should be able to record/store messages.

Hints

Looking at the files given, it must be obvious to you that you are given two solved introductory projects in the two .zip folders: One related to Audio Codec (echo back the microphone to speaker; would receive input from a microphone and output it straight away as feedback), and one related to memory (leds/switches used). Understand those two solved example designs as the first, important step. You have to figure out a way to use that audio input and instead of feeding back as a loop, have to store in memory replacing the values that were initially given by the switches. At the same time, instead of outputting the audio that was coming into the FPGA board, you would have to retrieve it from the memory.

1. Audio Codec:
   Read the general descriptions of the SSM2603 codec here (skip the specific FPGA board/Altera SoCKit board, Importing Pin Assignments, and Finding the Datasheets but you gain much information otherwise):

https://zhehaomao.com/blog/fpga/2014/01/15/sockit-8.html

FYI, the board datasheets are here:

a. Page 5 (copy/paste to browser):
https://reference.digilentinc.com/_media/anvyl:anvyl_sch.pdf

b. codec:
http://www.analog.com/media/en/technical-documentation/data-sheets/SSM2603.pdf

We have put a .zip file for ISE in which the projects are built for you for Anvyl board which we use in this course to test the audio on FPGA. Refer to readme.txt file in the .zip file as well. Note that the bitstream and the project itself are there. Also, reading the references mentioned above, you see that there are different clock frequencies needed for the codec. These are achieved by instantiating a PLL (the clock wizard HDL file under ipcore folder generated inside Xilinx) in the sockit_top.v for some clocks. In other words, the 100MHz clock is the input to the clock wizard and one 50 MHz and one 11.2896 MHz clock are outputs.

We will see later that the actual board oscillator clock is 100MHz which is given to the Ram Insterface block. The clock output of this block is 37.5MHz. You need to build a clock wizard yourself (different from the one that is already done for you inside the audio codec) to get two 100MHz clocks, one for the codec (used as the clock wizard of the codec as mentioned above) and one for picoblaze.

Also, pay attention that sample_req[1] and sample_end[1] signals allow you to correctly synchronize your code by waiting on each sample to be requested before trying to play each section of the message while also waiting on the sample to end in order to appropriately perform the recording of such audio input.

2. More info on the Audio Codec:
   The audio interface consists of a sockit_top module, an I2C controller, an audio codec module, and a module for audio effects. The sockit_top module combines all the components relating to audio codec to define what the audio interface is. It includes a clock wizard to synthesize two different frequencies, one from the main clock, and the other from the audio codec clock. The I2C controller is a serial bus protocol that provides communication between the FPGA board and the audio codec such that the board is the master, and the codec is the slave. The audio codec file is designed to convert serial communication to parallel communication for the audio effects module to handle. The module for audio effects can provide audio feedback for the memory to store, making it possible to store the recordings of the user through interacting with the switches.

3. Memory:
   The memory has 1Gbit of onboard DDR2 RAM. It is made up of an interface wrapper. To write to memory, the write enable signal must go high for data to be stored at a given address. To read data from memory, the read request signal must go high for the system to go to a provided address of where the data is located. Once the data is ready to be released, the data will be output. An acknowledge signal goes high as the controller accepts the data. Take advantage of the solved introductory project to understand the memory interface.

4. The top module of the design could contain the state controller for the memory as well as PicoBlaze. You could call to the memory, driver, UART, Audio codec, PicoBlaze, etc. PicoBlaze could be used to send the controls from the command console display by key

input and give this to the top module (you assemble your PSM file similar to what you did in Lab 6). The top module could contain two different always statements. The first could be for the memory module and contain all the states necessary to read, write, pause, select a song, and all the needed timing logic for storing and reading back legible audio for a specific amount of time. This entire state machine could run by the 37.5Mhz clock generated by the RAM itself. The second always statement could be controlled by a 100Mhz clock which is the same clock we use for the UART and PicoBlaze. This always statement could control the input and output data from our PicoBlaze assembly code.

5. "Ram_interface_guidelines.pdf" guidelines file and "ddr2_mem_test_ise_14.7_v1" .zip folder is also available. Read/write to memory is tested in this project which makes you familiar with the process. Refer to readme.txt file in the .zip file as well.

Note on clocks:

Please follow the steps below to resolve the clock buffer issue that may arise when connecting different modules in the final project:

1. The main clock signal (100 MHz) generated by the board (the crystal oscillator) should be used as the RAM interface wrapper interface's input clock.

2. The output clock signal (37.5 MHz) generated by the RAM interface wrapper should be used as input using the IP generator clocking wizard to generate two 100 MHz clock signals. Create a new source of IP core, search for clock wizard and select that. You will be passing the 37.5 Mhz clock of the Ram module output to the main clock input of this module, enter 37.5 into the primary clock value, Uncheck phase-alignment and IMPORTANT set source to Global Buffer and go to next screen, Create 1 output clock of 100 Mhz and go next, All optional items should be unchecked, Press next on each page changing nothing until end and select generate. Once created select it and go to View HDL Instiantion Template, from there copy paste the structural call into the users Top Module

IMPORTANT - The main clock from UCF file will be fed into the Ram modules main clock input ONLY, Take the 37.5Mhz clock it feeds out to the main input of the clock module we just created. The output of that 100Mhz clock will go to EVERYTHING else, except of course you will have to feed the 37.5 clock back into the system clock of the ram and the states controlled for the ram will need to be controlled by that 37.5 clock as well (clkout to sysclk).

3. Use these two 100 MHz clock signals as input clocks to the audio and Picoblaze modules, respectively.

Note on audio codec clock: Please do not discard the pll-clk_wiz_v3_6 file. Keep this file as it is inside the audio_sockit_top. This is actually the second clock wizard that is already created for you for the internal clocks of the codec. You can easily recreate the clock wizard for the codec yourself too. These are achieved by instantiating a PLL (the clock wizard HDL file under ipcore folder generated inside Xilinx) in the sockit_top.v for some clocks. In other words, the 100MHz clock is the input to the clock wizard and one 50 MHz and one 11.2896 MHz clock are outputs.

Relevance to Security, Safety, Environment, and Economic Factors: With respect to security and safety, this project addresses cryptographic approaches inherent in Xilinx ISE through safety/security of Advanced Encryption Standard. Safety and security measures can also be developed on Picoblaze softcore in addition to security and safety factors considered for bitstream security and safety mechanisms on the Spartan 6 FPGA board from Digilent. Students need to explore such factors in this project and understand the details. With respect to environmental factors, low-power and low-energy implementations through hardware and software co-design is explored. ASIC design

is the ultimate low-power and low-energy and environmental friendly, and after that FPGA and after that hard processor SW design. These factors will be considered in this project. Economic factors considered in this project includes estimates of the expenses needed to finish the project, effects of HW/SW co-design on savings in similar real-world project through considering economic and non-recurring engineering aspects.

Considering Global, Cultural, Social Factors: In this project, the students use Picoblaze and a terminal to build a GUI for pausing, deleting, adding, etc. a sound message. Also, they need to program the soft-processor in order to increase/decrease the playback volume using the FPGA board push buttons, DIP switches, etc. Students are given freedom to have these menus (for the GUI) either using English, or through commonly-used international/global abbreviations so that it is understandable in other languages. The GUI itself can be programmed so that students draw simple graphics, welcome messages, or directions instead of writing such information just in English. For such freedom, the ASCII codes are given to you (for letters, symbols, characters not necessarily in English, etc.) This ensures that the project is culturally diverse, if students choose to. For deaf, hard-of-hearing, or partially blind students, we have made the project such that in addition to hearing the voice of sound-tracks, the samples be stored in files so that tracking the file contents, students understand the deletions/additions. Moreover, for partially blind students, instead of using keyboard/monitor pairs, students can use push buttons, and hear the results without strictly being forced to create a visual GUI. The volume is controlled not through typing in English, but switches. Finally, global symbols can be used instead of English texts as part of the GUI (for example for rewind, FF, they can use >> or <<, or use other internationally-known symbols for playing, pausing, or volume

Deliverables and Demo (.Zip file submitted by the lead)

- **Project Demo**: Your team must record a Video demo and add to the report.
- **Project Report**: A project report template is posted on Canvas**.** The project report is due by **night of May 4**.
- **Add your project design files and project report (including the link to Demo) as .zip and upload.**

Grade

The grade distribution is based on (a) your report; and (b) the working features of your design including files. Total grade is 25 marks. You can get good partial mark if you can show you have put much effort into the project, and show some functionality. Therefore, do not get discouraged if you face obstacles here and there.

For the above, you will implement the user interface with Serial Terminal (PuTTY or similar interface) display, push-buttons, LEDs, and dip switches.

Note: The .zip file submission will be in the course section on Canvas only. You do not need to submit anything to the lab section Canvas for final project. We will use your marks for both lab/course though.